

Can Large Language Models Serve as Data Analysts? A Multi-Agent Assisted Approach for Qualitative Data Analysis

Zeeshan Rasheed^a, Muhammad Waseem^a, Aakash Ahmad¹, Kai-Kristian
Kemell^a, Wang Xiaofeng¹, Anh Nguyen Duc^b, Pekka Abrahamsson^a

Faculty of Computing and Communications, Lancaster University Leipzig

Faculty of Engineering, Free University of Bozen Bolzano

^a*Tampere University, Finland*

^b*University of South Eastern Norway, Norway*

Abstract

Context: Manual qualitative data analysis is time-intensive and can compromise validity and replicability, affecting analysis design, implementation, and reporting. Large Language Models (LLMs) enable human-bot collaboration in Software Engineering (SE), but their potential for qualitative data analysis in SE remains largely unexplored.

Objective: The objective of this study is to design and develop an LLM-based multi-agent system that synergizes human decision support with AI to automate various qualitative data analysis approaches.

Methods: We used LLM-based multi-agents systems to assist the qualitative data analysis process, deploying 27 agents, each responsible for a specific task, such as text summarization, initial code generation, and extracting themes and patterns.

Results: The main findings are: (1) the LLM-based multi-agent system accelerates the qualitative data analysis process, (2) the system effectively automates tasks such as text summarization, initial code generation, and theme extraction, and (3) the publicly accessible code facilitates validation and further evaluation.

Conclusion: The proposed LLM-based multi-agent system automates qualitative data analysis process, creating opportunities for researchers and practitioners. Future improvements focus on enhancing multilingual performance and integrating continuous expert feedback. The source code of pro-

posed system and system details can be found here: <https://github.com/GPT-Laboratory/Qualitative-Analysis-with-an-LLM-Based-Agentts>

Keywords: Large Language Models, Qualitative Data Analysis, Multi-Agent Systems, Automation, Empirical Software Engineering

1. Introduction

Large Language Models (LLMs) have transformed academia and various other fields by offering advanced capabilities in Natural Language Processing (NLP), automated content generation, and data analysis [1, 2, 3]. For instance, LLMs like Generative Pre-training Transformer (GPT) and Llama have shown capabilities in understanding, interpreting, and generating human-like text, making them crucial systems across various domains (e.g., sentiment analysis and document summarization) [4]. This technology has opened new directions in various domains of Software Engineering (SE) [5, 6]. The adoption of LLMs in SE is primarily driven by an innovative approach that transforms many SE tasks into activities involving data analysis and text classification. LLMs have shown promise in performing various SE tasks, including software development, text classification, and data analysis [7, 8, 9, 10, 11, 12].

In the domain of qualitative data analysis in empirical SE, LLMs can also play an important role [13]. LLMs can automate the extraction and interpretation of large volumes of text data [13]. For instance, the traditional approach to data analysis is highly dependent on human expertise, which consumes more time and human effort. Qualitative data analysis is dependent on primary data in the form of unstructured text or recordings from interviews. Software systems commonly employed for this purpose include MAXQDA [14], Nvivo [15], Atlas.ti [16], Dedoose [17], WebQDA [18], and QDAMiner [19]. However, these systems require human input, where researchers manually perform coding, interpret text, and make decisions. This process is time-consuming and requires a highly specific skill set and expertise to ensure the rigor of data analysis.

These traditional approaches to data analysis depend on human expertise, including survey instrumentation, statistical analysis, and interpretation. However, with LLMs, there is a paradigm shift towards more automated, intelligent systems [20]. Torii *et al.* [21] mentioned, LLMs have the potential to autonomously perform qualitative data analysis, such as analyz-

ing user feedback, software documentation, and development logs to extract valuable insights. By doing so, LLMs enhance the efficiency of data analysis and bring a level of depth to the interpretation of qualitative data, which was previously challenging to achieve [22]. Several researchers, including Chew *et al.* [20], Dai *et al.* [23], and Xiao *et al.* [13] explored the potential of LLMs in qualitative data analysis. However, their efforts primarily focus on automating thematic analysis and content analysis approaches to generate initial codes. Despite these advancements, a notable gap persists in the development of LLM-based systems that can autonomously perform all types of qualitative data analysis processes. Further research and development are required to address this gap and fully utilize the capabilities of LLMs within qualitative research methodologies.

In this paper, we propose an LLM-based multi-agent system to assist and automate the qualitative data analysis process, addressing challenges in scalability and efficiency. From the various qualitative research methods, we assist the five most popular methods used in SE: content analysis [24], thematic analysis [25], narrative analysis [26], grounded theory data analysis [27], and discourse analysis [28]. The proposed system analyzes large textual datasets and interview transcripts to autonomously perform qualitative data analysis. It integrates LLMs to help researchers handle diverse datasets and accelerate the analysis process with improved performance. Primary contributions of this study are:

- An LLM-based multi-agent system synergises human decision support with AI to automate various qualitative data analysis approaches, including thematic analysis, grounded theory, content analysis, narrative analysis, and discourse analysis.
- The integration of LLMs based system into the qualitative analysis process allows researchers to handle large and diverse datasets and speed up the process. This reduces reliance on manual processes, making data analysis more efficient and accessible.

Implication of research: The results of this research can provide foundations to researchers who can formulate new hypotheses about the role of LLMs and explore processes, patterns, and methods for LLM-driven data analysis. Software practitioners and data analysts can follow the reported results to experiment with delegating analytical tasks of data analysis to LLMs.

Structure of the paper: The rest of the paper is organized as follows. We review related work in Section 2 and describe the study methodology in Section 3. The results of this study are presented in Section 4. A discussion on key findings and their implications is provided in Section 5. The study concludes with future work in Section 6.

2. Background

In this section, we briefly present the background of the study with a focus on existing research. Section 2.1 provides an overview of studies concerning LLMs and SE. Section 2.2 provides a background study on qualitative data analysis in SE. Finally, section 2.3 examines works that have utilized LLMs for qualitative data analysis.

2.1. Large Language Models in Software Engineering

In recent years, LLMs have experienced a rapid advancement in various SE applications [29]. As Treude *et al.* [30] mentioned, the LLM and SE are interconnected through the application of NLP techniques to various tasks within the software development lifecycle. LLM’s language generation capabilities offer valuable assistance and enhancements to SE processes [31, 32]. The integration of LLMs into SE has marked a major transformation in this field [33]. LLMs have demonstrated valuable advantages over traditional methods like models guided by domain-specific languages, probabilistic grammars, and basic neural language models. Nowadays, LLMs have been applied to various field of SE. These include data analysis [34], text classification [35], software development [12], code search [36], unit test case generation [37], automated program repair [38], and many other.

Hou *et al.* [39] conducted a systematic review of 229 papers (2017–2023) on the use of LLMs in SE. They examined different LLM types, data processing techniques, performance optimization methods, and SE tasks where LLMs have been successfully applied, highlighting key challenges, research gaps, and future directions.

Zheng *et al.* [40] reviewed the integration of LLMs into SE, categorizing SE tasks into seven types and highlighting application examples, strengths, and limitations. They identified key challenges, including inconsistent LLM performance, limited evaluation of code-centric models, and the need for customized models tailored to specific SE tasks. Zheng *et al.* [41] examined the effectiveness of LLMs in the context of SE. They reviewed and evaluated

134 studies focused on code LLMs, highlighting the connections between code-specific LLMs and general-purpose LLMs. Furthermore, they conducted an in-depth analysis of how both general and code LLMs perform in various SE tasks, providing a detailed assessment of their capabilities across different sub-tasks.

Shin *et al.* [42] explored the capabilities of GPT-4 by integrating various prompting techniques (such as basic prompts, context learning, and task-specific prompts) to assess its performance on three common SE tasks: code generation, code summarization, and code translation. They compared GPT-4’s performance with 18 other fine-tuned LLMs. Additionally, Li *et al.* [43] demonstrated the effectiveness of fine-tuning and prompt engineering in automating code reviews, highlighting the role of context-rich prompts in improving model accuracy. Marvin *et al.* [44] highlighted the adaptability of prompt engineering across diverse SE applications, suggesting its potential to optimize resource allocation in LLM-driven workflows. Together, these studies underline the critical importance of combining personalized prompting techniques with fine-tuning to enhance LLM performance in varied SE contexts.

2.2. Qualitative Data Analysis in Empirical Software Engineering

Qualitative research is a type of research that focuses on collecting and analyzing non-numerical data (e.g., text, video, or audio) to understand concepts, opinions, or experiences [45]. As Seaman *et al.* [46] mentioned, qualitative methods are adopted and incorporated into SE research, which help researchers produce richer and more informative results. Qualitative data analysis in SE involves systematic examination of non-numerical elements like developer experiences, user feedback, design ideas, and processes efficiency[47]. Seaman *et al.* [46] introduces qualitative methods in empirical SE. In this paper, **Seaman** proposed that new research methods are important for examining non-technical aspects and that qualitative methods can be adapted and integrated into the design of empirical studies in SE. Motivated by inconsistent review standards, Dittrich *et al.* [48] proposed common evaluation criteria for qualitative research quality. In this paper, they proposed eight criteria that highlight the importance of clarity in the contribution of qualitative studies. Dittrich *et al.* also highlights the need for detailed procedures in qualitative data analysis to improve the understanding and interpretation of results.

Among the qualitative research methods, content analysis [24], thematic analysis [25], narrative analysis [26], grounded theory [27], and discourse analysis [28] are the most popular in empirical SE researchers [49], [24]. A content analysis extract the meaning of words or concepts and “goes beyond only counting words to analyze language extensively for the purpose of classifying big amounts of text into an effective number of categories with same meanings” [50]. According to Exton *et al.* [51], researchers use content analysis to quantify the usage of SE design patterns as a means of communication within empirical SE communities. Thematic analysis is very similar to content analysis. However, it differs in that the themes are typically not quantified [52]. Thematic analysis focuses on finding themes and also creating categories [25]. Narrative analysis is also a qualitative research method used to understand and interpret the stories people tell about their lives and experiences [53]. Catherine *et al.* [54] provided a overview of narrative methods in social research, cementing its place as a distinct qualitative method. Grounded theory starts with a question or an existing theory, followed by a thorough examination of the data. The data are analyzed through constant comparative analysis, where they are tagged with codes and grouped into concepts to identify themes [55]. Additionally, the data are analyzed during the collection process, with the analyzed data guiding further data collection [56]. Finally, discourse analysis is also a qualitative research method used to study written or spoken language in its social context, focusing on the ways language constructs meaning, identities, and power relations [57]. Zellig Harris [58] invented the term “discourse analysis ” in the 1970s, and his studies on face-to-face interaction laid foundational concepts [59]. In the 1980s and 1990s, scholars like Michel Foucault [60] and Norman Fairclough [61] expanded the field, linking discourse to power structures and social change, which led to the development of critical discourse analysis [62].

These qualitative research methods are also used to analyze the quality of SE studies [63], [26]. For instance, Stol *et al.* [64] reviewed 16 studies to evaluate the quality of SE studies using grounded theory. They concluded that many papers fail to generate a theory, do not clearly indicate which variant of grounded theory is used, and lack sufficient methodological details for rigorous evaluation. Additionally, the authors provided guidelines on how to conduct and report grounded theory studies effectively.

Table 1: Automation qualitative methods using AI techniques

S No	Paper Title	Automation Tasks	Reference
01	Cody: An AI-based system to semi-automate coding for qualitative research.	Automates coding through code rules and supervised ML	Rietz <i>et al.</i> [19]
02	LLM-assisted content analysis: using large language models to support deductive coding	Automate content analysis process with LLM	Chew <i>et al.</i> [20]
03	LLM-in-the-loop: leveraging large language model for thematic analysis	Automate thematic analysis with LLM	Dai <i>et al.</i> [23]
04	Automated thematic analysis of health information technology (hit) related incident reports	Automate thematic analysis method through NLP and ML	Li <i>et al.</i> [65]
05	Using LLMs for qualitative analysis can introduce serious bias	Identified the bias in LLMs when using for qualitative research	Ashwin <i>et al.</i> [66]
06	Developing and testing an automated qualitative assistant (AQUA) to support qualitative analysis	AQUA, an AI assistant, offers transparent and reliable coding of qualitative data	Lennon <i>et al.</i> [67]
07	Using ChatGPT for Thematic Analysis	Perform thematic analysis by using ChatGPT	Turobov <i>et al.</i> [68]
08	Automation of Qualitative Content Analysis: A Proposal	Semi automate content analysis by using ML techniques. Only automate the coding	Hoxtel <i>et al.</i> [69]
09	Semi-automated coding for qualitative research: A user-centered inquiry and initial prototypes	automate the tedious coding process, desire transparent systems that extend coding to large datasets	Marathe <i>et al.</i> [70]

2.3. Large Language Models in Qualitative Research

In recent studies, LLMs have demonstrated their capability in tasks similar to deductive coding, offering promising alternatives to traditional supervised learning methods [13]. These models excel in both zero-shot (no examples) and few-shot (few examples) learning scenarios, addressing some of the limitations inherent in conventional approaches [20]. Tornberg [71] observed that zero-shot GPT-4 annotations outperformed both crowd-workers and experts in terms of accuracy and inter-rater reliability when identifying the political affiliation of tweets from U.S. politicians. A study closely aligned with our research by Xiao *et al.* [13] and Dai *et al.* [23] explored the potential of LLMs in deductive coding. However they just automate the thematic analysis and content analysis approaches to generate initial codes. Their findings suggested that prompts based on existing codebooks were more effective than those with exemplar coding decisions, though both approaches fell short of expert coder performance. Building upon these foundations, our work extends the use of LLMs in qualitative research by employing a multi-agent system based on advanced LLMs to automate all types of qualitative data analysis, representing a step forward beyond existing applications in both scope and complexity. There is a lack of single platform that automate all aspects of qualitative analysis. This study aims to bridge this gap by introducing a platform that integrates LLMs with a multi-agent system. Our

approach aims to provide a versatile system that supports qualitative data analysis using LLM-based AI agents.

3. Research Method

This research aims to investigate the efficiency of LLMs to assist qualitative data analysis in SE. Our methodology is structured into two phases, each designed to test and analyze the capabilities of LLMs in this context. Below we discuss how our LLM-based multi-agent system performs qualitative analysis tasks.

3.1. Research Questions

Considering the objective of our study, we have formulated the following Research Questions (RQs):

RQ1. *How effective are LLM-based multi-agent systems to assist qualitative data analysis?*

The main aim of **RQ1** is to determine whether LLM-based multi-agent systems are capable of assisting in qualitative data analysis. This question was formulated to evaluate the ability of LLM-based agents to accurately interpret and analyze complex textual data without human intervention.

3.2. System Design

The section reports on how the proposed multi-agent system was developed to assist qualitative data analysis. In this project, we developed 27 agents, each of which is a specialized instance of an LLM, assigned a specific task to perform. As shown in Figure 1, the proposed multi-agent system receives diverse datasets from various sources. The multi-agent system processes the input and sends requests to the OpenAI API environment. Finally, the environment generates and returns a response. This is achieved by utilizing the capabilities of LLMs to understand and interpret complex language structures, making it possible to automate the various aspects of qualitative analysis such as thematic analysis, content analysis, narrative analysis, discourse analysis and grounded theory generation. For a more in-depth understanding, we refer to Table 2, showing the diverse datasets employed in our experiment and provides output in multiple formats, including a CSV file, an output area, and documents or PDF files. Below, we discuss the technical details of these agents.

Table 2: Input and output data-source

S.No	QDA Method	Input Data Source	Type of Dataset	Output Format
1	Thematic Analysis	Links	Github/StackOverflow Developer Discussion	CSV File
		Prompt		Doc file
		Upload Files		Output Box
		Voice Recording		
2	Content Analysis	Links	Customer Feedback	CSV File
		Prompt		Doc file
		Upload Files		Output Box
3	Narrative Analysis	Links	Blogs	CSV File
		Prompt	Human Biography	Doc file
		Upload Files		Output Box
4	Discourse Analysis	Links	Customer Reviews	CSV File
		Prompt	Online Conversation	Doc file
		Upload Files		Output Box
5	Grounded Theory	Links	Blogs	CSV File
		Prompt	Observations	Doc file
		Upload Files		Output Box

3.2.1. LLM Based Multi-Agent System

Firstly, we discuss the technical details of *algorithm 01*, which plays an important role in facilitating communication between agents and users. As shown in the proposed *algorithm 01*, the process begins by initializing the OpenAI API key. Once the API is initialized, the algorithm reads the input data provided by the user, which can consist of links, textual prompts, or even audio files. After the input is processed, the next step is to submit the input to the selected analysis method. For each selected type, the algorithm initializes conversation histories and assigns modular agents to perform specific tasks. Each agent performs a specific function, such as identifying themes, coding categories, or transcribing narratives, depending on the analysis selected. The agents communicate through a series of interactions with the OpenAI API, processing the input and generating the necessary insights. For audio inputs, transcription and processing are handled using a speech recognition module. Upon completing the analysis, the system parses the results and saves them in the format specified by the user, such as a CSV file or a JSON response. Below, we provide the technical details of each agent.

Thematic analysis is a method used in qualitative research to identify, analyze, and report patterns (themes) within data [72]. It minimally organizes and describes dataset in (rich) detail. It is usually applied to a set of texts. As we can see in Table 3, we developed five agents that work collaboratively to automatically perform thematic analysis on a given input. The *Agent Summary* is responsible for receiving the input. This agent pri-

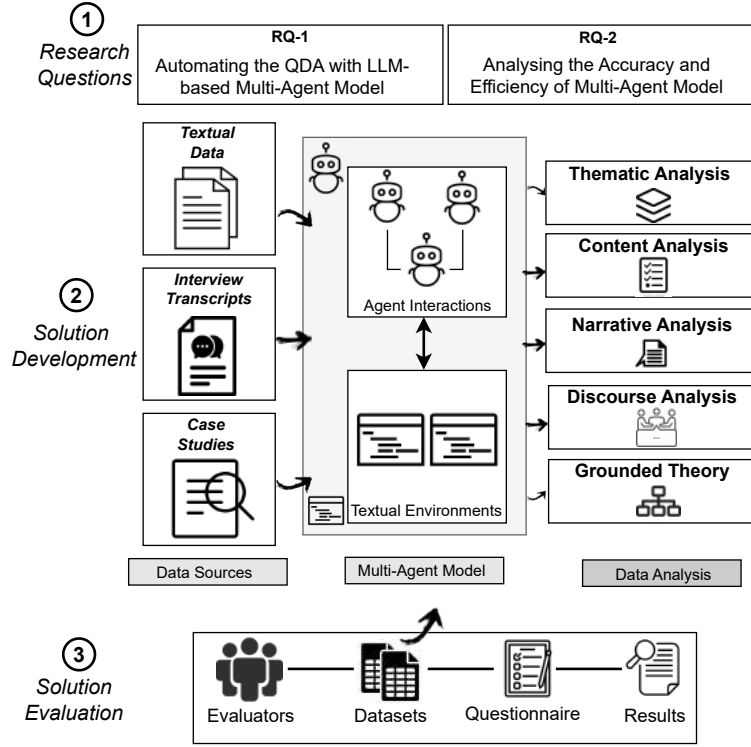


Figure 1: A workflow overview of the proposed system for automation of qualitative data analysis

primary task is to summarize the text and remove any unnecessary data. The first step is to configure scraper settings adapted to the structure of the discussion from the input data, ensuring the relevant information is captured efficiently. Following this, the algorithm loads and parses the input data to prepare it for summarization. The next step is to generate a clear and concise paragraph that extracts the required information from the input data. This summary is then formatted and compiled into a structured format and finally send the summarized data to *Agent Coders* for further action. The *Agent Coders* receive the summarized data from the *Agent Summary* and load the summarized data. The first step is to generate the initial codes to meet thematic requirements. These initial codes pass through a review process and make adjustment to ensure their accuracy and relevance. After

this review, the codes are compiled into a structured format. This structured approach ensures that the thematic analysis codes are systematically generated, reviewed, and prepared for further thematic analysis tasks. Finally, the finalized initial thematic codes are then transferred to *Agent Themes* for further processing. The *Agent Themes* main task is to generate themes for thematic analysis from initial thematic codes provided by *Agent Coders*. The process begins by receiving the initial thematic codes from *Agent Coders* and organizing these codes into preliminary themes, refining them to ensure consistency. After this refinement, the final themes are compiled into a structured format. These documented themes are then transferred to the final report for inclusion in the thematic analysis. This structured approach ensures that thematic codes are systematically organized and refined into meaningful themes that support thematic analysis. The next step is to verify the generated codes and themes. The *Agent Verify* receives the thematic codes and themes from *Agent Themes*. The process begins by initializing the verification phase, where verification parameters are configured based on quality standards. The themes and initial codes are then received and loaded for verification. The next step involves identifying and resolving any inconsistencies in the themes and codes to ensure accuracy. Once verified, the reviewed data is compiled into a structured format. The verification process and outcomes are documented thoroughly. The verified themes and codes are then transferred to the *Agent Finalize* for the final action. The *Agent Finalize* process starts by receiving the verified themes and codes and configuring the finalization parameters based on storage requirements and data formats. The verified data is then loaded and processed to ensure it matches the final output standards and formats. The finalized themes and codes are stored in the specified data format in the designated storage system. A final report is then generated, summarizing the themes, codes, and findings. The finalization process concludes with the termination of the process, ensuring that all data is accurately finalized, stored, and documented for future analysis.

Content analysis is used to analyze media interviews, open-ended surveys, and other forms of text data [73]. Researchers use content analysis to track changes over time, compare media content, understand the perspectives of different groups, or identify the prevalence of themes or perspectives in discussions. To automate the process of content analysis, we developed six AI agents. The *Agent Data Preprocessing* is the initial step in the autonomous content analysis process. This agent is tasked with summarizing the story, ensuring it is structured and cleaned for further processing by an-

Algorithm 1 Qualitative Data Analysis System with AI Agents

Require: Input data (e.g., project description, links, or audio)

Ensure: Finalized analysis results for each selected type (Thematic, Content, Narrative, Discourse, Grounded Theory)

```
1: Initialize OpenAI API:
2: Set the OpenAI_API_KEY with the appropriate API key.
3: Define the model (e.g., gpt-4o).
4: Prepare for Analysis:
5: Read the input_data from the user (links, prompts, audio).
6: Identify and select analysis types (Thematic, Content, Narrative, Discourse, Grounded Theory).
7: Initialize conversation_history for each selected analysis type.
8: Call Agents for Analysis:
9: for each selected_analysis_type do
10:   if Thematic Analysis is selected then
11:     Call the following agents:
12:     Agent Summary, Agent Coders,
13:     Agent Themes, Agent Verify,
14:     Agent Finalize.
15:   else if Content Analysis is selected then
16:     Call the following agents:
17:     Agent Data Pre-processing, Agent Codebook,
18:     Agent Coding Category, Agent Pattern Recognition,
19:     Agent Verify, Finalize Agent.
20:   else if Narrative Analysis is selected then
21:     Call the following agents:
22:     Agent Data Transcription, Agent Open Coding,
23:     Agent Axial Coding, Agent Selective Code,
24:     Agent Construct Narrative Analysis, Agent Finalize.
25:   else if Grounded Theory is selected then
26:     Call the following agents:
27:     Agent Data Scraper, Agent Coding,
28:     Agent Focused Coding, Agent Theoretical Coding,
29:     Agent Theory Development.
30:   else if Discourse Analysis is selected then
31:     Call the following agents:
32:     Data Transcribed Agent, Agent Themes,
33:     Agent Discourse Analyzer, Agent Contextual Analyzer,
34:     Agent Finalize.
35:   end if
36:   Initialize the conversation history for the selected agent(s).
37:   Call OpenAI API with the agent prompts.
38: end for
39: Agent Interaction:
40: for each round in interaction_rounds do
41:   Agent sends a message.
42:   Process the message to extract analysis results.
43:   Update the conversation history with the agent's response.
44: end for
45: Return Results:
46: Parse the final agent response.
47: Extract the results for each analysis type.
48: Save Processed Data:
49: if output_type is CSV then
50:   Save results to a CSV file.
51: else if output_type is JSON then
52:   Return response as JSON.
53: end if
54: For audio input, transcribe and process using speech_recognition.
```

other agent. The process start with data collection, where the user uploads input data from the client side. The preprocessing phase then summarizes the input data before passing it to the preprocessing module. Next, preprocessing settings are configured by defining parameters and criteria for identifying unnecessary words. The data cleaning phase involves identifying and removing unnecessary words and phrases from the input data. Finally, the cleaned and structured data is submitted to the *Agent Codebook*. The *Agent Codebook* transforms structured and cleaned data from the preparation phase into a finalized codebook. The process begins with the identification of initial codes based on the content of the data. These initial codes are then compiled into a single document to form the codebook. The next step involves validating the codebook to ensure consistency and clarity, ensuring that all codes are well-defined and applicable to the data. After validation, the codebook is finalized, making it ready for use. The final step is the transfer of the completed codebook to the *Agent Coding Category*, making it available for the coding team to apply in their analysis.

Table 3: The AI agent workflow to autonomously perform thematic analysis

Algorithm 1 Agent Summary	Algorithm 2 Agent Coders	Algorithm 3 Agent Themes
Input: Interview, textual data, case studies Output: Summarized input data and passed to Agent coders 1. Configure Scraper Settings: 2. scraper_settings $\leftarrow$ Configure scraper settings based on the discussion structure 3. content $\leftarrow$ Load and parse the input data pages 4. Create Summary: 5. summary $\leftarrow$ Create a clear and concise paragraph summarizing them 6. pass_to_agent_002 $\leftarrow$ Compile summaries into a structured format 7. End Process	Input: Summarized data from Agent Summary Output: Initial thematic analysis codes transferred to Agent Themes 1. Initialize Coding: 2. coding_parameters $\leftarrow$ Configure coding parameters based on thematic requirements 3. summarized_data $\leftarrow$ Receive and load summarized data from Agent Summary 4. Generate Initial Codes: 5. initial_codes $\leftarrow$ Extract and generate initial thematic codes from summarized_data using coding_parameters 6. review_initial_codes $\leftarrow$ Manually review and adjust the initial codes for accuracy and relevance 7. compiled_codes $\leftarrow$ Compile the reviewed and adjusted initial codes into a structured format 8. Transfer Codes: 9. send_to_agent_theme $\leftarrow$ Send the compiled initial thematic codes to Agent Themes for further action 10. End Process	Input: Initial thematic codes from Agent Coders Output: Coherent themes for thematic analysis 1. Initialize Theme Generation: 2. theme_parameters $\leftarrow$ Configure theme generation parameters based on analysis requirements 3. initial_codes $\leftarrow$ Receive and load initial thematic codes from Agent Coders 4. Organize Codes into Themes: 5. Refine and Consolidate Themes: 6. Compile Final Themes: 7. Transfer Themes: 8. send_to_final_report $\leftarrow$ Transfer the documented themes for inclusion in the final thematic analysis report 9. End Process
Algorithm 4 Agent Verify	Algorithm 5 Agent Finalize	
Input: Themes and initial codes from Agent Themes Output: Verified themes and codes stored in a specified format 1. Initialize Verification: 2. verification_parameters $\leftarrow$ Configure verification parameters based on quality standards 3. themes_and_codes $\leftarrow$ Receive and load themes and initial codes from Agent Themes 4. resolve_discrepancies $\leftarrow$ Identify and resolve any discrepancies or inconsistencies in the themes and codes 5. compiled_verified_data $\leftarrow$ Compile the verified and reviewed data into a structured format 6. Document Verification: 7. document_verification $\leftarrow$ Document the verification process and outcomes 8. Transfer Verified Data: 9. send_to_finalize_agent $\leftarrow$ Transfer the verified themes and codes to Finalize Agent for final action 10. End Process	Input: Verified themes and codes from Agent Verify Output: Finalized themes and codes stored in a specified format 1. Initialize Finalization: 2. finalization_parameters $\leftarrow$ Configure finalization parameters based on storage requirements and data formats 3. verified_data $\leftarrow$ Receive and load verified themes and codes from Agent Verify 4. finalized_data $\leftarrow$ Process verified data to ensure it conforms to final output standards and formats 5. Quality Assurance: 6. quality_check $\leftarrow$ Perform quality assurance check to ensure data integrity and formatting correctness 7. store_data $\leftarrow$ Store the finalized themes and codes in the specified data format in the designated storage system 8. Create Report: 9. create_final_report $\leftarrow$ Generate a final report summarizing the themes, codes, and findings 10. End Process	

The *Agent Coding Category* organizes and finalizes a categorized codebook for further use. The workflow begins with the input of a codebook from the *Agent Codebook* stage. The first step is to identify code categories based on the content of the codebook, followed by organizing these codes into their respective categories (categorized codes). Once categorized, the agent establishes relationships by describing how different categories relate to each other. These categorized codes are then compiled into structured documents (categorized codebook). Finally, the categorized codebook is ready to be transferred to the *Agent Pattern Recognition*. *Agent Pattern Recognition* convert a categorized codebook into a finalized pattern report for next agents. Initially, the agent extracts relevant data points from the categorized codebook. Furthermore, the agent finds initial patterns by extracting codes and analyze the relationship between the codes. Once these relationships are determined, the validation phase ensures the accuracy and relevance of the identified patterns. Detailed descriptions of these relationships and patterns are then documented. The process concludes with the generation of a detail pattern report, which provides a structured summary of the findings. The next step is to validates the pattern report generated by the *Pattern Recognition Agent*. Initially, the process involves reviewing the pattern report by documenting any potential issues that arise during the review. Following this, the agent verifies the identified patterns by cross-referencing them with the original data to confirm their validity. In the final step, the relationships identified in the patterns are validated through additional analysis. The process concludes with the distribution of a finalized validation report, which serves as a detail documentation for the next agent. The final step is to finalizes the validation report. The *Agent Finalize* reviews the validation feedback and suggests further refinements. Following this, the agent ensures the completeness and accuracy of the report by confirming that all data, patterns, and relationships are correctly represented. In the final step, the agent compiles and stores all the data in the final report.

Narrative analysis is a qualitative research method used to analyze personal stories, to understand how individuals make sense of events and actions in their lives [74]. To automate the narrative analysis approach, we developed six AI agents to understand the context and generate the narrative. The process starts with receiving the raw data such as audio recording or text. The *Agent Data Transcription* prepares data for transcription by confirming the quality of the text and audio to ensure data is enough for transcription. The main task involves transcribing the data, converting the

audio into written text. An initial quality check is conducted to remove unnecessary data and correct any evident transcription errors.

Upon verification, the transcribed text is compiled into a single document and then transferred for further processing. The *Agent Open Coding* receive the summarized data. The core task involves applying open codes to the data using inductive coding methods to generate codes directly from the text. These open codes are then documented to form the basis for a draft codebook. The agent proceeds by refining these codes, merging similar ones, and resolving any inconsistencies. This refined set of codes is compiled into an organized and clearly presented open coding report. Finally, the open coding report is transferred to *Agent Axial coding* for further steps in narrative analysis, completing this phase of the workflow. This structured approach ensures that raw transcription data is properly coded and documented.

The *Agent Axial Coding* begins with the collection of the open coding report generated by the *Agent Open Coding*. The initial step involves preparing for axial coding by confirming that the open coding report is properly formatted. The agent then identifies code relationships by determining categories and properties of the codes. Following this, axial codes are developed to describe the nature of these relationships. The final axial coding report ensures that the report clearly presents the axial codes and their relationships. Once completed, this report is transferred to *Agent Selective Code* for further analysis. This structured process enables a detailed understanding of the relationships between codes, which is essential for deeper narrative analysis in the following steps.

The *Agent Selective Coding* main aim is to generate selective code. The first step is to receive the axial coding report produced by the previous agent. The initial step is to identify core categories by determining the main categories from the axial codes. The agent then generates selective codes that integrate and refine axial codes under these core categories. The process continues with refining the selective codes by merging similar ones to ensure clarity and consistency. The refined selective codes and their relationships are then documented in a selective coding report. Finally, the selective coding report is transferred for use in next stages of analysis.

The *Agent Construct Narrative Analysis* receives the selective coding report produced by the *Agent Selective Coding*. The initial step involves preparing for narrative analysis by ensuring the selective coding report is properly formatted. Following this, the agent identifies key themes by determining the main themes that emerge from the data. The next step is to construct nar-

Table 4: The AI agent workflow to autonomously perform content analysis

Algorithm 1 Agent Data Preprocessing	Algorithm 2 Agent Codebook	Algorithm 3 Agent Coding Category
Input: User inputs data or uploads data from the client side. Output: Structured and cleaned data for the next agent. 01: Collect Data: 02: User inputs data or uploads data from the client side. 03: Preprocess Data: 04: summarized_data $\leftarrow$ Summarize the input data. 05: Pass the summarized data to the preprocessing module. 06: Configure Preprocessing Settings: 07: settings $\leftarrow$ Define parameters for data preprocessing. 08: criteria $\leftarrow$ Set criteria for identifying unnecessary words. 09: Clean Data: 10: unnecessary_words $\leftarrow$ Identify unnecessary words and phrases from the input data. 11: cleaned_data $\leftarrow$ Remove unnecessary words and phrases 12: Submit to Next Agent: 13: Pass the structured and cleaned data to the next agent for coding. End Process	Input: Summarized data Output: Finalized codebook for the coding team. 01: Define Initial Codes: 02: initial_codes $\leftarrow$ Identify initial codes based on the data content and research objectives. 03: Compile Codebook: 04: codebook $\leftarrow$ Compile initial_codes into a single document 05: Validate Codebook: 06: Ensure consistency and clarity in the codebook. 07: Finalize Codebook: 08: Transfere Codebook: 09: Transfer the codebook to the agent coding category End Process	Input: Comprehensive codebook from the agent codebook. Output: Finalized categorized codebook for the next agent. 01: Identify Code Categories: 02: categories $\leftarrow$ Identify categories based on codebook content. 03: categorized_codes $\leftarrow$ Organize codes into their respective categories. 04: Establish Relationships: 05: relationships $\leftarrow$ Describe how different categories relate to each other. 06: Compile Categorized Codebook 07: categorized_codebook $\leftarrow$ Compile categorized_codes into structure documents 08: Validate Categorized Codebook: 09: Ensure consistency and clarity in the categorized codebook. 10: Finalize Categorized Codebook: 11: Transfer Categorized Codebook End Process
Algorithm 4 Agent Pattern Recognition	Algorithm 5 Agent Verify	Algorithm 6 Finalize Agent
Input: Categorized codebook from the code categorization agent. Output: Finalized pattern report for the next agent. 01: Prepare Data for Analysis: 02: analysis_data $\leftarrow$ Extract relevant data points from the categorized codebook 03: Identify Initial Patterns 04: initial_patterns $\leftarrow$ Identify frequent co-occurrences and trends in the codes. 05: Analyze Relationships: 06: relationships $\leftarrow$ Determine direct and indirect relationships between codes 07: Validate Patterns 08: validated_patterns $\leftarrow$ Confirm the validity of identified patterns and clusters 09: relationship_documentation $\leftarrow$ Create detailed descriptions of relationships and patterns 10: Generate Pattern Report: 11: pattern_report $\leftarrow$ A structured document summarizing the findings End Process	Input: Pattern report from the pattern recognition agent. Output: Finalized validation report for the next agent. 01: Review Pattern Report: 02: review_notes $\leftarrow$ Document any observations or potential issues during the review. 03: Verify Identified Patterns: 04: verified_patterns $\leftarrow$ Confirm the validity of the patterns through cross-referencing with the original data. 05: Validate Relationships: 06: validated_relationships $\leftarrow$ Confirm the relationships through additional analysis or expert validation. 07: Distribute Validation Report: End Process	Input: Validation report from the validation and verification agent. Output: Finalized report for stakeholders and archiving 01: Review Validation Feedback: 02: review_notes $\leftarrow$ Document any final observations or areas needing further refinement. 03: Ensure Completeness and Accuracy: 04: completeness_check $\leftarrow$ Confirm that all data, patterns, and relationships are correctly represented. 05: Compile Final Report: 06: final_report $\leftarrow$ Create a comprehensive and polished document ready for dissemination End Process

rative segments by developing segments that logically flow and build on each other. The agent then analyzes the relationships and context by describing how different segments and themes relate to each other. This is followed by formulating analytical insights that provide a deeper understanding of the narrative. The process concludes with compiling a narrative analysis report that ensures the report is organized and clearly presents the analyzed narrative. The *Agent Finalize* workflow start with receiving the final narrative report generated by the *Agent Construct Narrative Analysis*. The initial task involves reviewing the final narrative report. Following this, the agent conducts a completeness check to ensure that no essential information is missing, followed by a consistency check to resolve any inconsistencies identified during the review. The next steps involve proofreading and editing to enhance readability and accuracy. Once revisions are complete, the final report is securely stored in an accessible location.

Ground theory involves in the construction of theories through the methodical gathering and analysis of data [75]. Grounded theory differs from other qualitative analysis methods as it is not dependent on a pre-existing theory to guide the direction of the research. It allows the researcher to develop a new theory that is grounded in the data that has been collected. To autonomously perform a grounded theory approach on our given input data, we developed five AI agents that work collaboratively to generate theory.

Table 5: The AI agent workflow to autonomously perform narrative analysis

Algorithm 01: Agent Data Transcription	Algorithm 02 Agent Open Coding	Algorithm 03 Agent Axial Coding
Input: Raw audio or video recordings from various sources. Output: Rough transcription compiled into a single document. 01: Collect Raw Data: 02: Prepare for Transcription: 03: preparation_check $\leftarrow$ Confirm the quality of the text and audio is adequate for transcription. 04: Transcribe Data: 05: transcribed_text $\leftarrow$ Convert the audio into written text. 06: Initial Quality Check: 07: quality_check $\leftarrow$ Correct any obvious errors or omissions in the transcription. 08: Compile Rough Transcription: 09: rough_transcription $\leftarrow$ Compile the transcribed text into a single document 10: Transfer Data 11: End Process	Input: Transcript data Output: Open coding report for further steps in narrative analysis. 01: Collect Rough Transcription: 02: Initial Review of Transcription 03: initial_notes $\leftarrow$ Document initial thoughts and observations. 04: Apply Open Codes: 05: open_codes $\leftarrow$ Use inductive coding to generate codes directly 06: Document Open Codes 07: codebook_draft $\leftarrow$ Begin compiling a draft codebook with initial codes 08: refined_codes $\leftarrow$ Merge similar codes, refine definitions, and resolve any discrepancies. 09: open_coding_report $\leftarrow$ Ensure the report is organized and clearly presents 10: Transfer Open Coding Report 11: End Process	Input: Open coding report Output: Axial coding report 01: Collect Open Coding Report 02: Prepare for Axial Coding 03: preparation_check $\leftarrow$ Confirm that the open coding report is properly formatted 04: Identify Code Relationships: 05: relationships_identified $\leftarrow$ Determine categories, properties, and dimensions of codes 06: Develop Axial Codes: 07: axial_codes $\leftarrow$ Generate codes that describe the nature of relationships 08: axial_codebook $\leftarrow$ Compile a codebook with axial codes 09: axial_coding_report $\leftarrow$ Ensure the report clearly presents the axial codes and the relationships 10: Transfer Axial Coding Report: 11: End Process
Algorithm 04 Agent Selective Code	Algorithm 05 Agent Construct Narrative Analysis	Algorithm 06 Agent Finalize
Input: Axial Coding Report Output: Selective Coding Report 01: Collect Axial Coding 02: Identify Core Categories: 03: core_categories $\leftarrow$ Determine the main categories 04: selective_codes $\leftarrow$ Generate codes that integrate and refine axial codes under core categories. 05: selective_codebook $\leftarrow$ Compile a codebook with selective codes 06: refined_selective_codes $\leftarrow$ Merge similar selective codes 07: Compile Selective Coding Report: 08: selective_coding_report $\leftarrow$ Ensure the report presents the selective codes and the relationships they describe. 09: Transfer Selective Code 10: End Process	Input: Selective coding report Output: Construct narrative analysis report 01: Collect Selective Coding Report: 02: Prepare for Narrative Analysis: 03: preparation_check $\leftarrow$ Confirm that the selective coding report is properly formatted 04: Identify Key Themes: 05: key_themes $\leftarrow$ Determine the main themes that emerge from the data. 06: Construct Narrative Segments: 07: narrative_segments $\leftarrow$ Develop segments that logically flow and build on each other. 08: Analyze Relationships and Context: 09: contextual_analysis $\leftarrow$ Describe how different segments and themes relate to each other 10: analytical_insights $\leftarrow$ Formulate insights that provide deeper understanding of the narrative 11: narrative_analysis_report $\leftarrow$ Ensure the report is organized and clearly presents the analyzed narrative 12: Transfer Analytic Report 13: End Process	Input: Final narrative report Output: Finalized narrative report 01: Receive Final Narrative Report 02: Review Final Narrative Report: 03: review_notes $\leftarrow$ Document any observations or areas needing further refinement. 04: completeness_check $\leftarrow$ Confirm that no crucial information is missing. 05: consistency_check $\leftarrow$ Address any inconsistencies found during the review. 06: Proofread and Edit: 07: edited_report $\leftarrow$ Make necessary edits to enhance readability and accuracy 08: formatted_report $\leftarrow$ Ensure the report is visually appealing and professionally presented 09: archive_location $\leftarrow$ Store the report in a secure and easily accessible location. 10: End Process

The first agent in this process is the *Agent Data Scraper*. The workflow of this agent start with the collection of raw data from various sources. The initial step involves preparing for transcription by identifying and documenting all sources of raw data. Following this, the agent cleans the data to ensure it is free from noise and inconsistencies. Once the data is cleaned, it is organized logically and consistently to form structured data. The next step involves compiling this cleaned and structured data into a well organized data report. Finally, the cleaned data report is transferred for further processing to the *Agent Coding*. *Agent Coding* main task is to generate the open coding from

given data. These open codes are then organized into broader categories to form a structured coding scheme. The agent then creates a draft codebook that includes all identified codes. Once the coding and codebook draft are finalized, the coding report is then transferred to *Agent Focused Coding* for further processing. This structured approach ensures that the cleaned data is properly coded and organized into meaningful categories. *Agent Focused Coding* workflow start with collecting the coding report produced by the *Agent Coding*. The initial step involves identifying codes by selecting those that are most relevant to the topic. Once the codes are identified, the agent applies focused coding to narrow down the data to its most essential elements. This process involves creating a codebook specifically for the focused codes. The focused coding report is then compiled to ensure that it clearly presents the focused codes. Finally, the focused coding report is transferred for further analysis to the *Agent Theoretical Coding*. The *Agent Theoretical Coding* main role is to identifying the main theoretical constructs from the focused codes. Once the theoretical constructs are identified, the agent generates codes that describe the relationships between these focused codes and the theoretical constructs. These relationships are then organized into a structured representation, forming the theoretical framework. A codebook is compiled specifically for these theoretical codes, and similar theoretical codes are merged and refined to ensure clarity and consistency. The agent then prepares a theoretical coding report that clearly presents the theoretical codes and their relationships. Finally, the theoretical coding report is transferred for further processing by subsequent agents. The *Agent Theory Development* first task is to determine the central constructs, referred to as core constructs, from the theoretical codes. Following this, the agent develops relationships by defining how these core constructs interact with one another. To validate the initial theory, the agent uses examples and data segments as supporting evidence. These relationships and constructs are then organized into a theoretical model that clearly illustrates the core constructs and their interactions. The agent compiles all findings into a theory development report that clearly presents the developed theory.

Discourse analysis is a qualitative and interpretive method used in linguistics, social sciences, and anthropology, among other fields, to study the ways in which language is used in context [76]. Researchers use this method to analyze the conversation in depth by examining any written or spoken text. In this approach, we used five agents to autonomously perform discourse analysis on the provided data.

The *Agent Data Transcribed* automates the initial stage of discourse analysis by transforming raw input data, such as spoken conversations, media content, and online communications, into a structured textual format. The workflow start by converting audio inputs to text by using speech-to-text APIs, and text inputs are refined for consistency. The agent then cleans the data by removing unnecessary elements, correcting errors, and ensuring terminology consistency. This processed data is summarized and stored in a structured format, making it ready for further analysis by subsequent agents.

Table 6: The AI agent workflow to autonomously perform ground theory

Algorithm 01 Agent Data Scraper	Algorithm 02 Agent Coding	Algorithm 03 Agent Focused Coding
Input: Raw data from various sources Output: Cleaned data report compiled and ready for further processing 01: Collect Raw Data: 02: Prepare for Transcription: 03: sources_identified $\leftarrow$ Identify and document all sources of raw data 04: Clean Data: 05: cleaned_data $\leftarrow$ Ensure that the data is free from noise and inconsistencies. 06: structured_data $\leftarrow$ Ensure the data is organized logically and consistently. 07: Compile Data Report: 08: data_report $\leftarrow$ Ensure the report is well-organized and cleaned data. 09: backup_created $\leftarrow$ Create a backup of the data 10: Transfer Data 11: End Process	Input: Cleaned data report from the data scraper agent. Output: Coding report 01: Collect Cleaned Data: 02: preparation_check $\leftarrow$ Confirm that the cleaned data report properly 03: Open Coding: 04: open_codes $\leftarrow$ Generate codes directly from the data 05: code_categories $\leftarrow$ Organize open codes into broader categories. 06: codebook_draft $\leftarrow$ Create a draft codebook with codes 07: refined_codes $\leftarrow$ Merge similar codes and refined it 08: Save and Backup 09: backup_created $\leftarrow$ Create a backup of the report and codebook 10: Transfer Coding Report 11: End Process	Input: Coding report Output: Focused coding report 01: Collect Coding Report 02: Identify Significant Codes: 03: significant_codes $\leftarrow$ Select codes that relevant to the topic 04: Apply Focused Coding: 05: focused_codes $\leftarrow$ Apply focused coding to narrow down the data to the most essential elements. 06: focused_codebook $\leftarrow$ Create a codebook specifically for the focused codes. 07: focused_coding_report $\leftarrow$ Ensure the report clearly presents the focused codes 08: backup_created $\leftarrow$ Create a backup of the report and codebook 09: Transfer Focused Coding Report: 10: End Process
Algorithm 04 Agent Theoretical Coding	Algorithm 05 Agent Theory Development	
Input: Focused coding report Output: Theoretical coding report and theoretical codebook 01: Collect Focused Coding 02: Identify Theoretical Constructs: 03: theoretical_constructs $\leftarrow$ Determine the main theoretical constructs 04: theoretical_codes $\leftarrow$ Generate codes that describe the relationships between focused codes and theoretical constructs 05: theoretical_framework $\leftarrow$ Create a structured representation of the theoretical constructs 06: theoretical_codebook $\leftarrow$ Compile a codebook specifically for the theoretical codes. 07: refined_theoretical_codes $\leftarrow$ Merge similar theoretical codes and refined it 08: theoretical_coding_report $\leftarrow$ Ensure the report clearly presents the theoretical codes 09: Transfer Theoretical Code 10: End Process	Input: Theoretical coding report Output: Theory development report 01: Collect Theoretical Coding Report: 02: core_constructs $\leftarrow$ Determine the central constructs 03: Develop Relationships 04: construct_relationships $\leftarrow$ Define how the core constructs interact 05: supporting_evidence $\leftarrow$ Use examples and data segments to validate the initial theory 06: theoretical_model $\leftarrow$ Ensure the model clearly illustrates the core constructs and their relationships. 07: theory_development_report $\leftarrow$ Ensure the report clearly presents the developed theory 08: backup_created $\leftarrow$ Create a backup of the report and model to prevent data loss. 09: End Process	

The *Agent Themes* automates theme identification in discourse analysis by processing cleaned and summarized data from the *Agent Data Transcribed*. The agent identifies themes by recognizing frequent terms and applying topic modeling techniques. The identified themes are then refined through a review process, including merging or splitting themes. Finally, the refined themes are saved in a structured format and transferred to the *Agent Discourse Analyser*, ensuring a robust foundation for subsequent discourse analysis stages. The *Agent Discourse Analyser* automates the analysis of discourse structure, organization, and linguistic features by processing themes identified by the

Agent Themes. The agent take input data from *Agent Themes* and analyzes paragraph organization by identifying topic sentences and supporting details, and evaluating sentence complexity and variety. The agent identifies linguistic features such as metaphors, similes, rhetorical devices, and stylistic elements. Word choice and lexical diversity are also analyzed. The results of the discourse analysis are stored in a structured format and transferred to the *Agent Contextual Analysis* for further processing.

Table 7: Table:7 The AI agent workflow to autonomously perform discourse analysis

Algorithm 01 Data Transcribed Agent	Algorithm 3 Agent Themes	Algorithm 3 Agent Discourse Analyser
Input: Spoken conversation, media content, online communication, etc. Output: Cleaned and summarized transcribed data. 1. Collect Input Data: 2. Preprocess Data: 3. extract_audio(audio_or_video_input) 4. transcriptions ← call_speech_to_text_api(audio_data) 5. else if input is text-based then 6. transcriptions ← preprocess_text_data(text_data) 7. Clean and Summarize Data: 8. Remove Unnecessary Data: 9. identify_and_remove_filler_words(transcriptions) 10. remove_irrelevant_sections(transcriptions) 11. correct_transcription_errors(transcriptions) 12. ensure_consistency_in_terminology(transcriptions) 13. Summarize Data: 14. cleaned_summarized_data ← summarize_transcriptions 15. save_transcriptions(cleaned_summarized_data, format="structured_format") 16. Transfer Data to Agent Themes: 17. End Process	Input: Cleaned and summarized transcribed data from Agent Transcribed. Output: Identified themes from the datasets. 1. Collect Cleaned Data: 2. Preprocess Data for Theme Identification: 3. tokenize_text(cleaned_data) 4. pos_tagging(cleaned_data) if necessary 5. Identify Themes: 6. frequent_terms ← identify_frequent_terms 7. potential_themes ← apply_topic_modeling(cleaned_data) 8. Refine Themes: 9. review_themes(themes) 10. merge_split_themes(themes) 11. validate_themes_with_experts(themes) if applicable 12. merge_split_themes(themes) 13. validate_themes_with_experts(themes) if applicable 14. Store Identified Themes: 15. save_themes(themes, format="structured_format") 16. Transfer Data to Agent Discourse Analyser 17. End Process	Input: Identified themes from Agent Themes. Output: Analysis of discourse structure, organization, and linguistic features. 1. Collect Thematic Data: 2. Receive identified themes from Agent Themes. 3. Preprocess Data for Discourse Analysis: 4. Analyze Paragraph Organization: 5. identify_topic_sentences_and_supporting_details 6. evaluate_sentence_complexity_and_variety(sentences) 7. Perform Sentiment Analysis: 8. sentiments ← use_sentiment_analysis_systems 9. classify_sentiments_at_sentence_and_paragraph_levels 10. Identify Linguistic Features: 11. detect_metaphors_and_similes(sentences) 12. identify_rhetorical_devices_and_stylistic_elements 13. analyze_word_choice_and_lexical_diversity 14. Store Analysis Results: 15. save_analysis_results(discourse_analysis_results, format="structured_format") 16. Transfer Data to Agent Contextual Analyser: 17. End Process
Algorithm 4 Agent Contextual Analyser	Algorithm 05 Agent Finaliser	
Input: Analysis of discourse structure. Output: Contextual analysis of the discourse. 1. Collect Discourse Analysis Results: 2. Identify Broader Contextual Elements: 3. identify_relevant_socio_cultural_factors(discourse_analysis_results) 4. Situational Context: 5. assess_immediate_situational_factors(analysis_results) 6. identify_roles_and_relationships_of_participants(discourse_analysis_results) 7. Integrate Contextual Information: 8. merge_contextual_elements_with_discourse_analysis 9. highlight_interplay_between_discourse_and_context 10. Perform Contextual Analysis: 11. analyze_contextual_influence_on_interpretation 12. identify_context_dependent_meanings_and_implications 13. Compile Contextual Analysis Results: 14. summarize_findings_for_each_contextual_element(contextual_analysis_results) 15. present_comprehensive_report_of_contextual_analysis 16. Store Contextual Analysis Results: 17. save_contextual_analysis_results(contextual_analysis_results, format="structured_format") 18. Transfer Data to Agent Finaliser: 19. End Process	Input: Contextual analysis results from Agent Contextual Analyser. Output: Finalized data ready for reporting or further use. 1. Collect Contextual Analysis Results: 2. ensure_completeness_and_accuracy(contextual_analysis_results) 3. validate_coherence_and_consistency(contextual_analysis_results) 4. verify_metadata_and_contextual_information(contextual_analysis_results) 5. Summarize Final Insights: 6. compile_key_findings(contextual_analysis_results) 7. highlight_important_themes(contextual_analysis_results) 8. prepare_executive_summaries(contextual_analysis_results) 9. Prepare Final Report: 10. organize_data_into_report(contextual_analysis_results) 11. include_visual_aids(contextual_analysis_results) 12. ensure_report_formatting(contextual_analysis_results) 13. Store Finalized Data and Report: 14. save_finalized_data_and_report(contextual_analysis_results, format="structured_format") 15. backup_data_and_report(contextual_analysis_results) 16. End Process	

The *Agent Contextual Analyser* automates the incorporation of contextual elements into discourse analysis. This agent begins by collecting the results of the discourse analysis and identifying broader contextual elements, such as socio-cultural factors and situational context. It assesses immediate situational factors and identifies roles and relationships of participants to integrate contextual information with the discourse analysis. The agent merges these elements to highlight the interplay between discourse and context, and performs a detailed contextual analysis to understand the influence of context on interpretation, meanings, and implications. The contextual analysis re-

sults are compiled, summarized, and presented in a report. These results are then stored in a structured format and transferred to the *Agent Finaliser*, ensuring a thorough understanding of the discourse within its broader context. The *Agent Finaliser* is the concluding component in the automated discourse analysis process. It synthesizes and finalizes the contextual analysis results provided by the *Agent Contextual Analyser*. It validates the coherence and consistency of the results and verifies metadata and contextual information. The agent then summarizes final insights, highlighting key findings and important themes, and prepares executive summaries. The finalized data and report are stored in a structured format.

4. Results

In this section, we present the study results of the proposed LLM-based multi-agent system for qualitative data analysis. Below, we present the results of our LLM-based proposed system in Section 4.1, specifically reporting the outcomes of RQ1.

4.1. Effectiveness of LLM-based Multi Agent System (RQ1)

Our proposed LLM-based multi-agent system assist traditional qualitative data analysis. Our goal is to integrate the LLM-based agents into qualitative data analysis to process and analyze large, diverse datasets and test their effectiveness for qualitative data analysis. The results indicate that integrating LLMs into qualitative research accelerates the analysis process and improves performance. However, certain limitations remain, emphasizing the need for further improvements in their application. Below, we have provided the results of various types of qualitative data analysis.

The proposed LLM-based multi-agent system has been successfully implemented and tested for autonomous qualitative data analysis. First, we demonstrate our proposed system’s result for **Thematic Analysis**.

As shown in Figure 2, we present a demonstration of our proposed system for thematic analysis. As depicted, we input GitHub links and select thematic analysis to prompt the system to identify issues from the text and perform thematic analysis. The generated output is then obtained in a CSV file. The system can handle various input formats. Users can choose their preferred method of input, allowing for a flexible and user-friendly interaction. Furthermore, we incorporate text extracted from Stack Overflow into our prompt. Specifically, we opt for thematic analysis to guide the system

in identifying the cause from the provided text. As illustrated in Figure 2, the process begins by summarizing the text and identifying the cause. Subsequently, the system proceeds to generate initial codes, which are then broken down into subcategories and categories. This time we get final results in output box. This systematic approach allows for a comprehensive and structured analysis of the input text. In this project, users have the autonomy to define and articulate the specific analysis they wish to perform. This streamlined process allows for efficient and automated exploration of the underlying concepts and themes within the given data, showcasing the system’s ability to deliver insightful outcomes without manual intervention. Our results demonstrate the system’s capability to autonomously execute qualitative data analysis methods on diverse datasets, streamlining the analysis process and reducing the need for manual intervention. In our proposed system, we have implemented an additional feature allowing users to upload text, document (doc), and Portable Document Format (pdf) files as input. The system subsequently autonomously generates responses by activating specific qualitative analysis approaches, thereby facilitating a comprehensive examination of the provided data. This functionality extends the utility of our proposed system, making it accessible for practitioners and researchers alike. Moreover, the system is versatile enough to be employed for interview analysis, enabling practitioners and researchers to utilize it for conducting interviews and summarizing the interview text efficiently

Next, we present the results of the proposed system for **Narrative Analysis**. First, we provide the input to our proposed system, which then generates a narrative analysis. The first step is to summarize the text, followed by extracting open coding from the summarized data. In this step, the system breaks down the data into discrete parts. During open coding, the data is labeled with codes that describe what is happening in the text, aiming to uncover key points and their meanings. Second, the system autonomously performs axial coding on the open coding data. Axial coding follows open coding and involves connecting categories to subcategories, linking them through relationships and patterns. This process helps refine and reorganize the initial codes into a coherent structure that shows how different concepts interact with each other. The third step is to autonomously perform selective coding. In this step, the system identifies the core category that integrates all other categories. Finally, the proposed system refines the analysis to form a narrative or theory that best explains the phenomenon under study. By validating the generated response, we observe that the narrative constructs

produced by the proposed system lack creativity and holistic storytelling, which a human analyst could provide, making the output feel mechanical.

TUNI Luottamuksellinen - Confidential (3Y)

Submit Links

Thematic
Content
Discourse
Ground
Narratives

Issue
Cause
Solution
Pattern
Themes

Input Link

Enter links here separated by commas

Text classification

Submit

Download CSV

Chose QDA method, then select option from the displayed box

Output File

A	B	C	D
Identified Issue	Thematic coding for identified issue	Subcategory	Category
Inconsistency in styling across different browsers	Styling inconsistency across browsers	Browser compatibility	Styling
Issues with implementing Tailwind CSS in a specific project	Implementation challenges	Project-specific implementation challenges	Technical challenges
The TailwindCSS plugin is not working with a specific version of a framework	Incompatibility issue	Compatibility issues	Technical issue
Unwanted Margin on Mobile View	Unwanted Margin on Mobile View	Mobile View	UI/UX

Thematic
Content
Discourse
Ground
Narrative

Issue
Cause
Solution
Pattern
Themes

Submit

Output File

Based on the discussion on the GitHub page, the identified cause for the issue is an incompatibility issue with the `autoprefixer` package. It seems that updating the `autoprefixer` package resolves the problem, as well as optionally updating the `tailwind` and `postcss` packages to avoid any compatibility issues.

Identified cause:
Incompatibility with `autoprefixer` package

Thematic coding for identified cause:
•Dependency issue
•Compatibility issue

Sub category for thematic coding of identified cause:
•Dependency management
•Package compatibility

Category for sub category of identified cause:
•Technical issue/bug

Figure 2: Generated result by proposed system

We now present the results of our proposed system for autonomously performing **Content Analysis**. First the proposed system initially summarizes the given input dataset and then autonomously generates a codebook for the summarized data. Next, the system performs coding categorization based on the initially generated codebook. Finally, the system autonomously generates pattern recognition based on the created categories. The system reduces bias in thematic categorization and pattern recognition, providing an impartial overview of the feedback. On the other hand, the quality of the generated analysis depends heavily on the clarity and consistency of the input data. Poorly structured or ambiguous feedback lead to inaccurate or incomplete insights.

Discourse Analysis focuses on how language constructs and conveys meaning in different contexts. Initially, we provided input and then proposed system first transcribed the data and summarized it for further processing. The next step was to generate themes based on the summarized data. Finally, the proposed system performed a discourse analysis based on the generated themes, helping to understand in detail how these themes are

presented, how language is used to shape their meaning and how they resonate with different audiences. LLM-based system effectively extracts and classifies various discourse patterns, such as frustration, satisfaction, fraud, or negligence. However, there is a strong chance that the system may misinterpret context, such as sarcasm, cultural references, or subtle implications, which a human analyst would recognize.

Ground Theory involves in the construction of theories through the analysis of data. The system first summarized the content to highlight essential information. It then proceeded to perform initial open coding, breaking down the raw data into meaningful codes that represented various concepts and patterns. Following this, the system perform focused coding, where it break down the open codes to focus on the most relevant and essential elements of the data, refining the analysis. After focused coding, the system moved on to theoretical coding, where it generated relationships between the focused codes and developed constructs, forming a structured theoretical concept that connected the central themes identified during the analysis. Finally, the system produced a theory development report, showing how the core constructs interacted and validating the theory with supporting evidence from the dataset.

5. Discussion

In this paper, we proposed LLM based multi agent system to assist qualitative analysis process. The multi-agent system interprets a wide range of textual and audio data and autonomously performs various types of qualitative data analysis.

LLM-Based Multi-Agent System for Qualitative Analysis (RQ1): The results indicate that integrating LLM based multi agents into qualitative analysis is representing a step forward towards automation of big data analysis. Furthermore, we believe that our findings have several implications for the future practice of related research methods. Firstly, the implementation of a multi-agent system that interprets vast quantities of textual and audio data highlights the potential of AI in enhancing the efficiency of qualitative research. Furthermore, our proposed system opens new opportunities for establishing a new standard in data interpretation and analysis within both industry and academia. Integrating LLM technology into qualitative analysis accelerates the data analysis process, reduces the manual effort and time required for practitioners and researchers to analyze large datasets, and

ensures high accuracy. This work has broader implications, particularly for large-scale industries. Furthermore, it has the potential to reduce the costs for qualitative studies, as the system is capable of handling complex analytical tasks independently.

However, despite these implications, there are still some challenges that need to be addressed in the future. Firstly, we utilized LLMs to perform qualitative data analysis. However, regarding data privacy, certain ethical concerns were raised, particularly related to the potential exposure of sensitive information and the ethical implications of using LLMs for data analysis. Although we designed specific prompts for each step in qualitative analysis that effectively generated the output to meet our objectives, it is important to acknowledge that we cannot claim these prompts to be the most optimal or accurate. According to Wei *et al.* [77], there is still a gap in exploring the potential of prompts to extract better outputs from LLMs. We believe that improved versions of prompts may lead to better outcomes. Finally, we only utilized OpenAI API, while many others LLM are available. Therefore, we cannot guarantee that using other LLMs will achieve comparable results. Future work could apply this framework to other open-source LLMs, such as LLaMA [78] and Falcon [79]. However, the computational resources and cost might be the potential limitations. We also acknowledge that the proposed system provides autonomous analysis, the use of LLMs poses potential privacy concerns, especially if sensitive qualitative data are processed. The study did not extensively address data governance or privacy protocols, which could be a limitation when deploying the system in practice. We also incorporated a feedback mechanism that enables the system to learn from user input, allowing for continuous improvement in both performance and accuracy.

Our future goal is to involve a more diverse set of participants, including industrial experts, business analysts, and market researchers from various fields where qualitative data analysis plays a critical role. We also plan to utilize benchmarks to broaden the evaluation of our proposed system. Additionally, investigating the model’s performance in multilingual settings could broaden its applicability in global research contexts.

6. Conclusions

In this paper, we introduce an LLM-based multi-agent system designed to assist various types of qualitative data analysis, including thematic analysis,

content analysis, narrative analysis, discourse analysis, and grounded theory. The main objective is to test the ability of LLM for qualitative data analysis. The initial results of the proposed system indicate that it autonomously performs the analysis on the given dataset. However, there is still a need to highlight the importance of ongoing refinement to address potential areas for improvement.

Our future goal is to focus on exploring the system’s performance in multi-lingual settings to extend its applicability in diverse global research contexts. Additionally, we will continue to emphasize the importance of maintaining a feedback loop with domain experts to ensure the ongoing refinement and enhancement of the system’s capabilities.

7. Acknowledgment

This project is co-funded by the European Union and Business Finland under project BF/Amalia-2023/SW.

References

- [1] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 2023.
- [2] G Bharathi Mohan, R Prasanna Kumar, P Vishal Krishh, A Keerthi-nathan, G Lavanya, Meka Kavya Uma Meghana, Sheba Sulthana, and Srinath Doss. An analysis of large language models: their impact and potential applications. *Knowledge and Information Systems*, pages 1–24, 2024.
- [3] Jesse G Meyer, Ryan J Urbanowicz, Patrick CN Martin, Karen O’Connor, Ruowang Li, Pei-Chen Peng, Tiffani J Bright, Nicholas Tatonetti, Kyoung Jae Won, Graciela Gonzalez-Hernandez, et al. Chat-gpt and large language models in academia: opportunities and challenges. *BioData Mining*, 16(1):20, 2023.
- [4] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 1469–1481. IEEE, 2023.

- [5] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. Software testing with large language model: Survey, landscape, and vision. *arXiv preprint arXiv:2307.07221*, 2023.
- [6] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M Zhang. Large language models for software engineering: Survey and open problems. In *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pages 31–53. IEEE, 2023.
- [7] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- [8] Yihan Cao, Siyu Li, Yixin Liu, Zhiling Yan, Yutong Dai, Philip S Yu, and Lichao Sun. A comprehensive survey of ai-generated content (aigc): A history of generative ai from gan to chatgpt. *arXiv preprint arXiv:2303.04226*, 2023.
- [9] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590*, 2023.
- [10] James Finnie-Ansley, Paul Denny, Brett A Becker, Andrew Luxton-Reilly, and James Prather. The robots are coming: Exploring the implications of openai codex on introductory programming. In *Proceedings of the 24th Australasian Computing Education Conference*, pages 10–19, 2022.
- [11] Aakash Ahmad, Muhammad Waseem, Peng Liang, Mahdi Fahmideh, Mst Shamima Aktar, and Tommi Mikkonen. Towards human-bot collaborative software architecting with chatgpt. In *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, pages 279–285, 2023.
- [12] Zeeshan Rasheed, Muhammad Waseem, Kai-Kristian Kemell, Wang Xiaofeng, Anh Nguyen Duc, Kari Systä, and Pekka Abrahamsson. Autonomous agents in software development: A vision paper. *arXiv preprint arXiv:2311.18440*, 2023.

- [13] Ziang Xiao, Xingdi Yuan, Q Vera Liao, Rania Abdelghani, and Pierre-Yves Oudeyer. Supporting qualitative analysis with large language models: Combining codebook with gpt-3 for deductive coding. In *Companion Proceedings of the 28th International Conference on Intelligent User Interfaces*, pages 75–78, 2023.
- [14] Udo Kuckartz and Stefan Rädiker. *Analyzing qualitative data with MAXQDA*. Springer, 2019.
- [15] AlYahmady Hamed Hilal and Saleh Said Alabri. Using nvivo for data analysis in qualitative research. *International interdisciplinary journal of education*, 2(2):181–186, 2013.
- [16] Brigitte Smit. Atlas. ti for qualitative data analysis. *Perspectives in education*, 20(3):65–75, 2002.
- [17] Michelle Salmona, Eli Lieber, and Dan Kaczynski. *Qualitative and mixed methods data analysis using Dedoose: A practical approach for research across the social sciences*. Sage Publications, 2019.
- [18] António Pedro Costa, Francislê Neri de Souza, António Moreira, and Dayse Neri de Souza. webqda 2.0 versus webqda 3.0: a comparative study about usability of qualitative data analysis software. In *Developments and Advances in Intelligent Systems and Applications*, pages 229–240. Springer, 2018.
- [19] Tim Rietz and Alexander Maedche. Cody: An ai-based system to semi-automate coding for qualitative research. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–14, 2021.
- [20] Robert Chew, John Bollenbacher, Michael Wenger, Jessica Speer, and Annice Kim. Llm-assisted content analysis: Using large language models to support deductive coding. *arXiv preprint arXiv:2306.14924*, 2023.
- [21] Maya Grace Torii, Takahito Murakami, and Yoichi Ochiai. Expanding horizons in hci research through llm-driven qualitative analysis. *arXiv preprint arXiv:2401.04138*, 2024.

- [22] John Roberts, Max Baker, and Jane Andrew. Artificial intelligence and qualitative research: The promise and perils of large language model (llm)‘assistance’. *Critical Perspectives on Accounting*, 99:102722, 2024.
- [23] Shih-Chieh Dai, Aiping Xiong, and Lun-Wei Ku. Llm-in-the-loop: Leveraging large language model for thematic analysis. *arXiv preprint arXiv:2310.15100*, 2023.
- [24] Joanna F DeFranco and Phillip A Laplante. A content analysis process for qualitative software engineering research. *Innovations in Systems and Software Engineering*, 13:129–141, 2017.
- [25] Virginia Braun and Victoria Clarke. Using thematic analysis in psychology. *Qualitative research in psychology*, 3(2):77–101, 2006.
- [26] Nadia N Kellam, Karen Sweeney Gerow, and Joachim Walther. Narrative analysis in engineering education research: Exploring ways of constructing narratives to have resonance with the reader and critical research implications. In *2015 ASEE Annual Conference & Exposition*, pages 26–1184, 2015.
- [27] Barney Glaser and Anselm Strauss. *Discovery of grounded theory: Strategies for qualitative research*. Routledge, 2017.
- [28] Jonathan Potter. Discourse analysis. *Handbook of data analysis*, pages 607–624, 2004.
- [29] Yunhe Feng, Sreecharan Vanam, Manasa Cherukupally, Weijian Zheng, Meikang Qiu, and Haihua Chen. Investigating code generation performance of chat-gpt with crowdsourcing social data. In *Proceedings of the 47th IEEE Computer Software and Applications Conference*, pages 1–10, 2023.
- [30] Christoph Treude. Navigating complexity in software engineering: A prototype for comparing gpt-n solutions. *arXiv preprint arXiv:2301.12169*, 2023.
- [31] Jonas Thiergart, Stefan Huber, and Thomas Übellacker. Understanding emails and drafting responses—an approach using gpt-3. *arXiv preprint arXiv:2102.03062*, 2021.

- [32] Adam Hörnemalm. Chatgpt as a software development tool: The future of development, 2023.
- [33] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. Learning to represent programs with graphs. *arXiv preprint arXiv:1711.00740*, 2017.
- [34] Jack W Rae, Sebastian Borgeaud, Trevor Cai, Katie Millican, Jordan Hoffmann, Francis Song, John Aslanides, Sarah Henderson, Roman Ring, Susannah Young, et al. Scaling language models: Methods, analysis & insights from training gopher. *arXiv preprint arXiv:2112.11446*, 2021.
- [35] Youngjin Chae and Thomas Davidson. Large language models for text classification: From zero-shot learning to fine-tuning. *Open Science Foundation*, 2023.
- [36] Xiaodong Gu, Hongyu Zhang, and Sunghun Kim. Deep code search. In *Proceedings of the 40th International Conference on Software Engineering*, pages 933–944, 2018.
- [37] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617*, 2020.
- [38] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.
- [39] Federico Quin, Danny Weyns, Matthias Galster, and Camila Costa Silva. A/b testing: a systematic literature review. *Journal of Systems and Software*, page 112011, 2024.
- [40] Zibin Zheng, Kaiwen Ning, Jiachi Chen, Yanlin Wang, Wenqing Chen, Lianghong Guo, and Weicheng Wang. Towards an understanding of large language models in software engineering tasks. *arXiv preprint arXiv:2308.11396*, 2023.
- [41] Zibin Zheng, Kaiwen Ning, Yanlin Wang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. A survey of large language models for

- code: Evolution, benchmarking, and future trends. *arXiv preprint arXiv:2311.10372*, 2023.
- [42] Jiho Shin, Clark Tang, Tahmineh Mohati, Maleknaz Nayebi, Song Wang, and Hadi Hemmati. Prompt engineering or fine tuning: An empirical assessment of large language models in automated software engineering tasks. *arXiv preprint arXiv:2310.10508*, 2023.
 - [43] Youjia Li, Jianjun Shi, and Zheng Zhang. An approach for rapid source code development based on chatgpt and prompt engineering. *IEEE Access*, 2024.
 - [44] Ggaliwango Marvin, Nakayiza Hellen, Daudi Jjingo, and Joyce Nakatumba-Nabende. Prompt engineering in large language models. In *International conference on data intelligence and cognitive informatics*, pages 387–402. Springer, 2023.
 - [45] Julia Bailey. First steps in qualitative data analysis: transcribing. *Family practice*, 25(2):127–131, 2008.
 - [46] Carolyn B. Seaman. Qualitative methods in empirical studies of software engineering. *IEEE Transactions on software engineering*, 25(4):557–572, 1999.
 - [47] Terhi Kilamo, Marko Leppänen, and Tommi Mikkonen. The social developer: now, then, and tomorrow. In *Proceedings of the 7th International Workshop on Social Software Engineering*, pages 41–48, 2015.
 - [48] Yvonne Dittrich, Michael John, Janice Singer, and Bjørnar Tessem. Editorial for the special issue on qualitative software engineering research. *Information and software technology*, 49(6):531–539, 2007.
 - [49] Steve Adolph, Philippe Kruchten, and Wendy Hall. Reconciling perspectives: A grounded theory of how people manage the process of software development. *Journal of Systems and Software*, 85(6):1269–1286, 2012.
 - [50] Azizah Ahmad, Rafidah Abd Razak, Mohd Syazwan Abdullah, Wan Rozaini Sheik Osman, Abdul Bashah Mat Ali, and Abdul Razak Rahmat. Business intelligence model for sustainability of the malaysian rural telecenters. *Journal of Southeast Asian Research*, 2011, 2011.

- [51] Christopher Exton. The role of content analysis in the development of theory and understanding of software engineering. In *12 International Workshop on Software Technology and Engineering Practice (STEP'04)*, pages 5–pp. IEEE, 2004.
- [52] Aaron Ahuvia. Traditional, interpretive, and reception based content analyses: Improving the ability of content analysis to address issues of pragmatic and theoretical concern. *Social indicators research*, 54:139–172, 2001.
- [53] Martin Cortazzi. Narrative analysis. *Language teaching*, 27(3):157–170, 1994.
- [54] Chtherine Kohler Rissman. Narrative analysis: Qualitative research methods, 1993.
- [55] Hsiu-Fang Hsieh and Sarah E Shannon. Three approaches to qualitative content analysis. *Qualitative health research*, 15(9):1277–1288, 2005.
- [56] Ji Young Cho and Eun-Hee Lee. Reducing confusion about grounded theory and qualitative content analysis: Similarities and differences. *Qualitative report*, 19(32), 2014.
- [57] Rosalind Gill. Discourse analysis. *Qualitative researching with text, image and sound*, 1:172–190, 2000.
- [58] Zellig S Harris and Zellig S Harris. *Discourse analysis*. Springer, 1970.
- [59] Ellen F Prince. Discourse analysis in the framework of zellig s. harris. *Current Trends in Textlinguistics*, page 191, 1978.
- [60] Heinz Steinert. The development of" discipline" according to michel foucault: Discourse analysis vs. social history. *Crime and Social Justice*, (20):83–98, 1983.
- [61] Norman Fairclough. Discourse and text: Linguistic and intertextual analysis within discourse analysis. *Discourse & society*, 3(2):193–217, 1992.
- [62] Norman Fairclough. Critical discourse analysis. In *The Routledge handbook of discourse analysis*, pages 9–20. Routledge, 2013.

- [63] James Thomas and Angela Harden. Methods for the thematic synthesis of qualitative research in systematic reviews. *BMC medical research methodology*, 8:1–10, 2008.
- [64] Klaas-Jan Stol, Paul Ralph, and Brian Fitzgerald. Grounded theory in software engineering research: a critical review and guidelines. In *Proceedings of the 38th International conference on software engineering*, pages 120–131, 2016.
- [65] Yanyan Li, Casper Shyr, Elizabeth M Borycki, and Andre W Kushniruk. Automated thematic analysis of health information technology (hit) related incident reports. *Knowledge Management & E-Learning*, 13(4):408, 2021.
- [66] Julian Ashwin, Aditya Chhabra, and Vijayendra Rao. Using large language models for qualitative analysis can introduce serious bias. *arXiv preprint arXiv:2309.17147*, 2023.
- [67] Robert P Lennon, Robbie Fraleigh, Lauren J Van Scoy, Aparna Keshaviah, Xindi C Hu, Bethany L Snyder, Erin L Miller, William A Calo, Aleksandra E Zgierska, and Christopher Griffin. Developing and testing an automated qualitative assistant (aqua) to support qualitative analysis. *Family medicine and community health*, 9(Suppl 1), 2021.
- [68] Aleksei Turobov, Diane Coyle, and Verity Harding. Using chatgpt for thematic analysis. *arXiv preprint arXiv:2405.08828*, 2024.
- [69] Annette Hoxtell. Automation of qualitative content analysis: A proposal. In *Forum: Qualitative Social Research*, volume 20. Freie Universität Berlin, 2019.
- [70] Megh Marathe and Kentaro Toyama. Semi-automated coding for qualitative research: A user-centered inquiry and initial prototypes. In *Proceedings of the 2018 CHI conference on human factors in computing systems*, pages 1–12, 2018.
- [71] Petter Törnberg. Chatgpt-4 outperforms experts and crowd workers in annotating political twitter messages with zero-shot learning. *arXiv preprint arXiv:2304.06588*, 2023.

- [72] Ashley Castleberry and Amanda Nolen. Thematic analysis of qualitative research data: Is it as easy as it sounds? *Currents in pharmacy teaching and learning*, 10(6):807–815, 2018.
- [73] James W Drisko and Tina Maschi. *Content analysis*. Pocket Guide to Social Work Re, 2016.
- [74] Sarah Earthy and Ann Cronin. Narrative analysis. In *Researching social life*. Sage, 2008.
- [75] Julianne S Oktay. *Grounded theory*. Pocket Guide to Social Work Re, 2012.
- [76] Barbara Johnstone and Jennifer Andrus. *Discourse analysis*. John Wiley & Sons, 2024.
- [77] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [78] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [79] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Merouane Debbah, Etienne Goffinet, Daniel Heslow, Julien Launay, Quentin Malartic, et al. Falcon-40b: an open large language model with state-of-the-art performance, 2023.